# 计算机学院

# 并行程序设计实验报告

# Lab1 CPU 架构编程

Cache 与超标量对程序性能的影响

姓名：柯云超

学号：2413575

专业：计算机科学与技术

学院：计算机学院

2026年 3月 20日

# 目录

**1 实验任务** **2**

**2 实验环境与测试方法** **2**
2.1 实验环境 . . . . . 2
2.2 测试数据与规模设计 . . . . . 2
2.3 计时方式 . . . . . 3

**3 算法设计与优化思路** **3**
3.1 矩阵列内积 . . . . . 3
3.2 求和 . . . . . 3

**4 实验结果与分析** **3**
4.1 矩阵实验 . . . . . 3
4.2 求和实验 . . . . . 4
4.3 汇编观察与 profiling 结果 . . . . . 5

**5 结论** **6**

# 1 实验任务

本次实验包含两个问题:

1. 给定 $n \times n$ 矩阵 $B$ 和向量 $a$，计算每一列与向量的内积,

$$s_j = \sum_{i=0}^{n-1} B_{i,j} a_i.$$

   对比逐列访问的平凡算法和面向 cache 的优化算法。

2. 给定 $n$ 个浮点数，计算其和,

$$S = \sum_{i=0}^{n-1} x_i.$$

   对比链式累加与适合超标量执行的多链累加、两两归约等实现。

本次提交的代码没有继续使用单文件结构，而是按功能拆分为多个源文件：计时与 profiling 框架位于 `src/benchmark.*`，矩阵实验位于 `src/matrix_experiment.*`，求和实验位于 `src/sum_experiment.*`，环境信息与 profiling 输出分别位于 `src/environment.*` 和 `src/profile.*`，入口文件为 `src/main.cpp`。这种组织方式更方便把不同优化版本并列管理，也便于后续继续扩展 SIMD 或多线程版本。

# 2 实验环境与测试方法

## 2.1 实验环境

| 项目 | 配置 |
|---|---|
| CPU | Intel Core i9-13900H |
| L1d Cache | 48 KiB |
| L2 Cache | 1280 KiB |
| L3 Cache | 24576 KiB |
| 操作系统 | Ubuntu 22.04.5 LTS on WSL2 |
| 编译器 | g++ 12.3.0 |
| 编译选项 | `-std=c++17 -O3 -march=native` |
| 额外控制 | `-fno-tree-vectorize -fno-tree-slp-vectorize`，绑定 CPU0 |

表 1: 实验平台配置

关闭自动向量化是为了尽量把实验重点保留在“cache 局部性”和“指令级并行”本身，而不是让编译器自动生成 SIMD 指令掩盖差异。

## 2.2 测试数据与规模设计

两部分实验的数据都使用固定公式生成，便于复现实验并检查正确性:

$$a_i = 0.25 + ((17i + 11) \bmod 97)/97.0,$$

$$B_{i,j} = 0.1 + ((131i + 17j) \bmod 251)/251.0.$$

矩阵实验选取 $n = \{64, 128, 256, 384, 512, 768, 1024, 1536, 2048\}$，对应工作集大小为 $8n^2$ Byte。其中 $n = 384$ 时矩阵大小约为 1.125 MiB，已经接近本机单核 L2；$n = 2048$ 时矩阵大小达到 32 MiB，已经超过共享 L3。

求和实验选取 $n = \{2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}, 2^{22}, 2^{23}\}$，数组大小为 $8n$ Byte，用于观察从 cache 友好到带宽受限的变化过程。

### 2.3 计时方式

程序使用 `std::chrono::steady_clock` 计时。对于每组规模，不预先固定测试次数，而是反复执行待测函数直到总计时超过 250 ms，再用总时间除以执行次数得到单次平均时间。这样做能显著降低小规模测试中过短运行时间带来的误差。每次测试前先进行两轮 warm-up，并将线程绑定到逻辑核 0，减小调度波动。

## 3 算法设计与优化思路

### 3.1 矩阵列内积

矩阵部分实现了三个版本：

1. **平凡算法**：外层按列循环，内层按行跳跃访问 `matrix[row * n + col]`。该写法与 C/C++ 的行主存储不一致，跨行访问步长大，空间局部性较差。
2. **行优先算法**：把循环次序改成“按行遍历矩阵，再把该行对所有列的贡献累加到输出向量”。这一改动不改变计算复杂度，但让内层访问连续内存，cache line 利用率明显更高。
3. **行优先 + 4 路展开**：在行优先基础上，对内层循环做 4 路手动展开，减少循环控制开销，进一步给处理器提供更稳定的指令流。

矩阵优化的关键不是减少乘加次数，而是让访问顺序匹配行主存储布局。第三个版本属于在此基础上的局部改良，收益通常小于“访问顺序修正”本身，但在中等规模数据上仍然能带来可见提升。

### 3.2 求和

求和部分实现了四个版本：

1. **链式累加**：单个累加器顺序求和，结构最简单，但依赖链最长。
2. **两路独立累加**：将一条依赖链拆成两条独立链，末尾合并结果。
3. **四路独立累加**：继续把依赖链拆成四条，进一步提高指令级并行度。
4. **两两归约**：先把相邻元素两两相加，再对中间结果继续归约，直到只剩一个结果。该方法依赖深度更短，但需要反复读写中间结果。

这里的优化重点是“减少串行依赖”。链式累加的每次加法都依赖上一次结果，而两路、四路独立累加把累加操作分散到多个寄存器上，更容易被超标量流水线并行发射。两两归约理论上也能缩短依赖深度，但中间数组的访存成本可能抵消收益。

## 4 实验结果与分析

### 4.1 矩阵实验

表 2 中，加速比 1 表示“平凡算法 / 行优先”，加速比 2 表示“平凡算法 / 行优先+4 路展开”。

| $n$ | 平凡/ms | 行优先/ms | 行优先+U4/ms | 加速比1 | 加速比2 |
|---|---|---|---|---|---|
| 64 | 0.002379 | 0.001731 | 0.001223 | 1.37 | 1.95 |
| 128 | 0.014189 | 0.007140 | 0.005770 | 1.99 | 2.46 |
| 256 | 0.062870 | 0.027437 | 0.022990 | 2.29 | 2.73 |
| 384 | 0.158872 | 0.061481 | 0.052697 | 2.58 | 3.01 |
| 512 | 0.884909 | 0.105562 | 0.093286 | 8.38 | 9.49 |
| 768 | 1.445654 | 0.243319 | 0.214320 | 5.94 | 6.75 |
| 1024 | 5.237621 | 0.505124 | 0.422654 | 10.37 | 12.39 |
| 1536 | 19.342532 | 1.876966 | 1.807951 | 10.31 | 10.70 |
| 2048 | 31.570206 | 3.178951 | 2.974866 | 9.93 | 10.61 |

表 2: 矩阵列内积实验结果

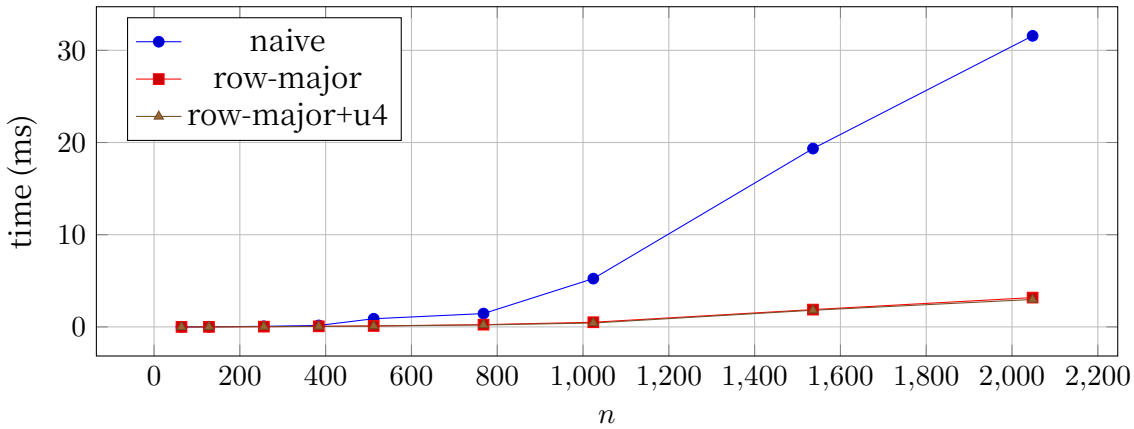


图 1: 矩阵列内积三种实现的运行时间

矩阵实验的趋势比较清楚:

1. 当 $n \leq 384$ 时，三种实现的差距还不算特别夸张，加速比主要在 2 到 3 倍之间。此时工作集仍然接近 L2 范围，虽然逐列访问不理想，但尚未完全失控。
2. 当 $n = 512$ 时，矩阵规模达到 2 MiB，已经明显超过 1280 KiB 的 L2，平凡算法时间陡增，而行优先算法仍然保持较平稳增长，加速比突然跃升到 8 倍以上。
3. 在 $n = 1024$ 时，行优先 + 4 路展开取得了本次矩阵实验最高加速比 12.39 倍。这里主要收益仍来自访问顺序修正，循环展开只是在此基础上再减少了一部分循环控制开销。
4. 当 $n$ 增长到 1536 和 2048 后，矩阵规模逼近甚至超过 L3，三种实现都继续变慢，但优化版本仍然稳定保持约 10 倍量级的优势，说明访存模式依然是决定性因素。

## 4.2 求和实验

表 3 中，三列加速比都以链式累加为基准。

| $n$ | 链式/ms | 两路/ms | 四路/ms | 归约/ms | 两路加速比 | 四路加速比 |
|---|---|---|---|---|---|---|
| 1024 | 0.000616 | 0.000333 | 0.000171 | 0.000533 | 1.85 | 3.61 |
| 4096 | 0.002708 | 0.001341 | 0.000744 | 0.002043 | 2.02 | 3.64 |
| 16384 | 0.010566 | 0.005477 | 0.003521 | 0.010409 | 1.93 | 3.00 |
| 65536 | 0.047305 | 0.022644 | 0.014536 | 0.043753 | 2.09 | 3.25 |
| 262144 | 0.181432 | 0.092731 | 0.065947 | 0.338249 | 1.96 | 2.75 |
| 1048576 | 0.705655 | 0.379622 | 0.305291 | 1.618618 | 1.86 | 2.31 |
| 4194304 | 3.664419 | 2.697514 | 2.930423 | 7.991180 | 1.36 | 1.25 |
| 8388608 | 7.133793 | 5.286948 | 5.101690 | 16.656185 | 1.35 | 1.40 |

表 3: 求和实验结果

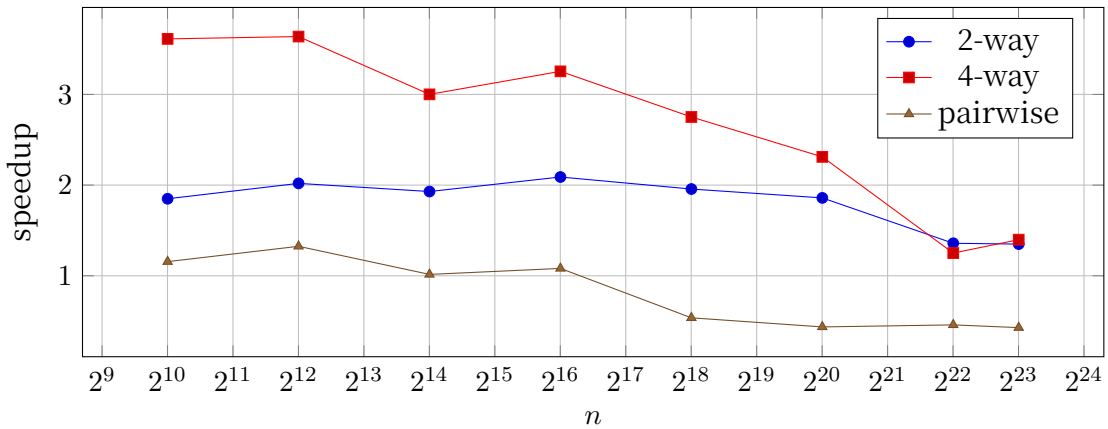


图 2: 求和优化版本相对链式累加的加速比

求和实验说明了超标量优化和访存开销之间的取舍：

1. 两路和四路独立累加在小规模数据上效果非常明显。四路版本在 $n = 4096$ 时达到 3.64 倍加速，说明拆分依赖链确实能显著提高处理器同时发射加法指令的能力。
2. 两路版本的提升较稳，但始终不如四路版本。这个现象很自然，因为四路版本提供了更多互不依赖的累加链，给调度器留下了更大余地。
3. 当 $n$ 增大到 $2^{22}$ 和 $2^{23}$ 时，数组大小达到 32 MiB 和 64 MiB，已经越过 L3 范围，四路版本的加速比下降到约 1.3 到 1.4 倍。此时瓶颈已从“依赖链长度”逐渐转向“内存带宽”。
4. 两两归约只在很小规模时略优于链式累加，规模继续增大后明显落后。原因在于它虽然缩短了依赖深度，但需要不断写回中间结果，访存代价过高。

另外，多链累加和归约都会改变浮点加法顺序，因此与链式累加存在微小数值差异。实测误差量级在 $10^{-13}$ 到 $10^{-10}$ 之间，属于浮点舍入误差的正常范围。

## 4.3 汇编观察与 profiling 结果

从 `objdump -Cd -Mintel` 提取的汇编可以看到，链式累加的关键循环只维护一个累加寄存器：

Listing 1: 链式累加与多链累加的关键汇编片段

```
sum_naive:
    vaddsd xmm0, xmm0, QWORD PTR [rdi]

sum_superscalar2:
    vaddsd xmm0, xmm0, QWORD PTR [rax]
    vaddsd xmm1, xmm1, QWORD PTR [rax+0x8]

sum_superscalar4:
    vaddsd xmm0, xmm0, QWORD PTR [rax]
    vaddsd xmm3, xmm3, QWORD PTR [rax+0x8]
    vaddsd xmm1, xmm1, QWORD PTR [rax-0x10]
    vaddsd xmm2, xmm2, QWORD PTR [rax-0x8]
```

这与时间结果是吻合的：`sum_naive` 只有一条依赖链，而两路、四路版本分别引入了两个和四个独立累加寄存器，使得加法指令更容易并行发射。

按照实验指导书的建议，我还保留了 `perf_event_open` 的 profiling 代码，尝试记录 cycles、instructions 和 cache miss。但当前环境是 WSL2，本机直接访问硬件计数器失败，`results/profile_results.csv` 中所有项目都返回了 `perf_event_open is unavailable in this environment`。因此，本报告对底层原因的判断主要依据两类证据：一是不同工作集规模下的时间变化趋势，二是反汇编得到的目标代码结构。

# 5 结论

本次实验完成了两个 CPU 架构相关问题的实现、优化和测试，可以得到以下结论：

1. 对矩阵列内积问题，真正决定性能的是访存模式。把逐列访问改成行优先访问后，加速比很快随着工作集越过 L2/L3 边界而增大，本机最高达到 12.39 倍。

2. 对求和问题，多链累加确实能够有效利用超标量特性。四路独立累加在小规模数据上达到 3.64 倍加速，但随着数据规模变大，收益会受到内存系统限制而收敛。

3. 两两归约虽然在理论上减少了依赖深度，但在当前实现中引入了额外的中间结果读写，因此整体并不占优。这说明“减少依赖”并不自动等价于“更快”，还要结合访存代价综合判断。

综合来看，本次实验比较直观地说明了现代 CPU 性能并不只由计算量决定。数据访问顺序、依赖链长度以及编译后机器指令的结构，都会直接影响最终运行时间。